! Supervised Learning



© 2023 Olivier Claerbout

Lesmateriaal

- Via GitHub account:
  git clone https://github.com/oclaerbout/syntra_data_scientist.git
- © 2023 Olivier Claerbout

  Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!

© 2023 Olivier Claerbout

Overzicht

- Boosting
- AdaBoost (Adaptive Boosting) Theorie
- AdaBoost: voorbeeld
- GradientBoosting Theorie
- Gradient Boosting: voorbeeld

© 2023 Olivier Claerbout

Boosting

© 2023 Olivier Claerbout

Boosting

- Gaan verder op concepten van Decision Trees
- Boosting is een andere methode om de decision tree te verbeteren
- Gradient boosting was een idee van Leo Breiman

- Interessante artikels:
  - *wikipedia.org/wiki/Boosting_(machine_learning)*
  - *wikipedia.org/wiki/AdaBoost*

© 2023 *Olivier Claerbout*

Boosting

- Boosting is een methodologie die wordt toegepast op een bestaand ML algoritme, vaak op een decision tree. Het is dus geen ML algoritme op zich.
- Het is een Ensemble Techniek die meerdere zwakke leeralgoritmen combineert om een sterk leeralgoritme te vormen
- We spreken van meta-learning

! *Een zwakke leerfunctie is eenvoudigweg een model dat iets beter presteert dan willekeurig gokken.*

© 2023 *Olivier Claerbout*

Boosting

- Algemene formule voor boosting:

$$F_t(x) = \sum_{t=1}^{T} f_t(x) \qquad \text{met} \qquad f_t(x) = \alpha_t h(x)$$

Kan in theorie elk ml algoritme zijn!

De gecombineerde sterke leerfunctie is de som van alle zwakke leerfuncties over T iteraties.

Elke zwakke leerfunctie op zich is het resultaat van het toekennen van een coëfficiënt (gewicht) α_t aan de t-de zwakke leerfunctie h(x). Deze coëfficiënt wordt bepaald in functie van de nauwkeurigheid van h(x), waarbij beter presterende 'zwakke' leerders een hoger gewicht krijgen.

We gaan er dus van uit dat een combinatie van zwakke leerfuncties h(x) met elk een coëfficiënt α samen een effectieve ensemble schatter kunnen vormen.

Is dit wel zo?

© 2023 Olivier Claerbout

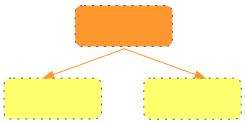
AdaBoost – Adaptive Boosting

- AdaBoost gebruikt een ensemble van zwakke leerders en combineert hen via een gewogen som
- Adaboost gebruikt de fouten (misclassificaties) van de reeds gemaakte zwakke leerders om de volgende nieuwe zwakke leerder bij te sturen
- De resultaten van de laatste tree worden gebruikt om de volgende tree bij te sturen – vandaar adaptive boosting!
- Trainen gebeurt dus serieel itt parallel bij random forest!

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Wat is een zwakke leerder?
  - Een model wat te eenvoudig is om op zichzelf goed te presteren
  - De zwakste decision tree noemen we een stump: één node en twee bladeren
  - Dit is de weak learner waar we mee gaan werken in AdaBoost



© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

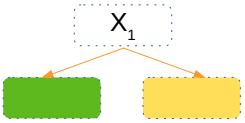
- Slow learning versus hard fitting
  - Een enkele decision tree gaat alle data in één keer fitten. Dit is een harde fit.
  - AdaBoost daarentegen combineert meerdere weak learners, waardoor het ensemble model traag kan leren van de features.
  - Laat ons dit vanuit een voorbeeld bekijken

© 2023 Olivier Claerbout

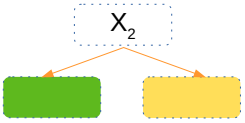
AdaBoost – Adaptive Boosting

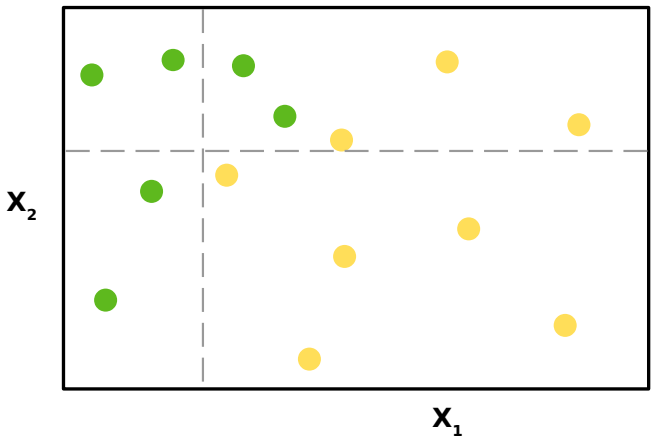
- We hebben een classificatie taak

Hoe zou een stump classificatie er uitzien?



OF:

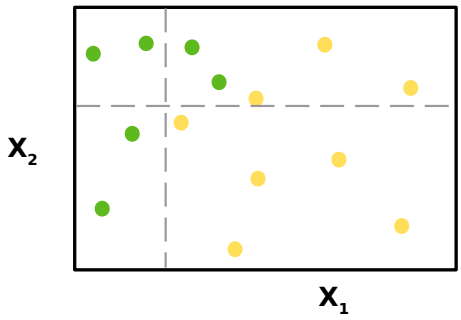




© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

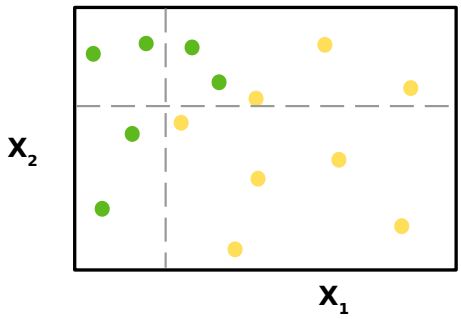
- Wat we dus willen doen is meerdere stumps combineren, er een gewicht aan toekennen en ze combineren tot een krachtiger model.
- Hoe kunnen we deze stumps combineren?
- Hoe kunnen we de performantie verbeteren met een ensemble, een ‘metamodel’?



© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Het AdaBoost proces
  - Formules
  - Algoritmische stappen
  - Visueel overzicht van deze stappen

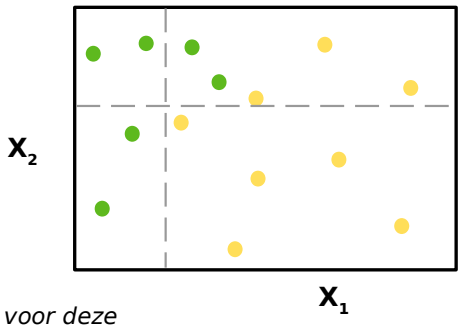


© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Formules
  - $F_t(x) = \sum_{t=1}^{T} f_t(x)$ (zie eerder; T = aantal weak learners – seriële stumps)

$f_t(x)$ Is een weak learner: een stump



$f_t(x) = \alpha_t h(x)$

α 'gewicht' parameter; t betekent voor deze specifieke learner. Goede stump: hoge α

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Formules

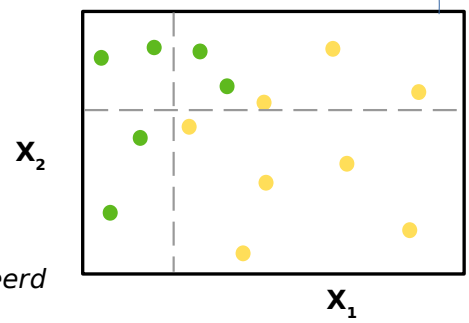
$$F_t(x)=\sum_{t=1}^{T} f_t(x) \qquad f_t(x) \qquad f_t(x)=\alpha_t h(x)$$

We gaan ook telkens de weights van elk datapunt i aanpassen met een gewicht w_t voor elke tree → $w_{i,t}$

$$E_t=\sum_i E[F_{t-1}(x_i)+\alpha_t h(x_i)]$$

Bepalen van α om de fout E_t te minimaliseren

E[ ] : foutfunctie die wordt geminimaliseerd



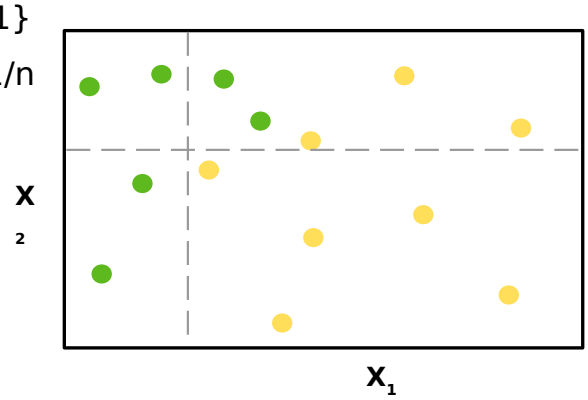
E_t: totale fout in 'stage' t; error term die we willen minimaliseren door α

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- De algoritmische stappen
  - We starten met datapunten x_1 … x_n
  - De gewenste outputs zijn y_1 … y_n, y ϵ {-1, 1}
  - Initiële gewichten $w_{1,1}$ … $w_{n,1}$ ingesteld op 1/n
  - Error functie E(f(x), y, i) = $e^{-y_i f(x_i)}$
  - Zwakke leerfunctie output h:x → {-1, 1}

Output is -1 of 1!



© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Verloop van de training
  - Voor t in 1 … T (voor elke tree!):
    - Vind de zwakke learner $h_t(x)$ die de error ϵ_t minimaliseert, de gewogen som error voor de fout geclassificeerde punten

$$\epsilon_t = \sum_{i=1, h_t(x_i) \neq y_i}^{n} w_{i,t}$$

  - Kies $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
  - Voeg toe aan het ensemble $F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$

© 2023 Olivier Claerbout

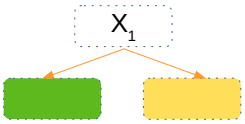
AdaBoost – Adaptive Boosting

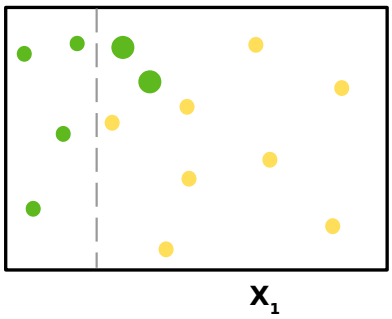
- Verloop van de training
  - Update de weights: $w_{i,t+1} = w_{i,t} e^{-y_i \alpha_t h_t(x_i)}$ voor i in 1 … n
  - Normaliseer $w_{i,t+1}$ zodat $\sum_i w_{i,t+1} = 1$

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Visualisatie van de stappen:
  - We kiezen een zwakke leerfunctie $h_t(x)$ die ϵ_t *(gewogen som misgeclassificeerde punten)* minimaliseert





  - Update het gewicht van de punten waarbij de misgeclassificeerde punten zwaarder wegen
  - We bepalen $\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$

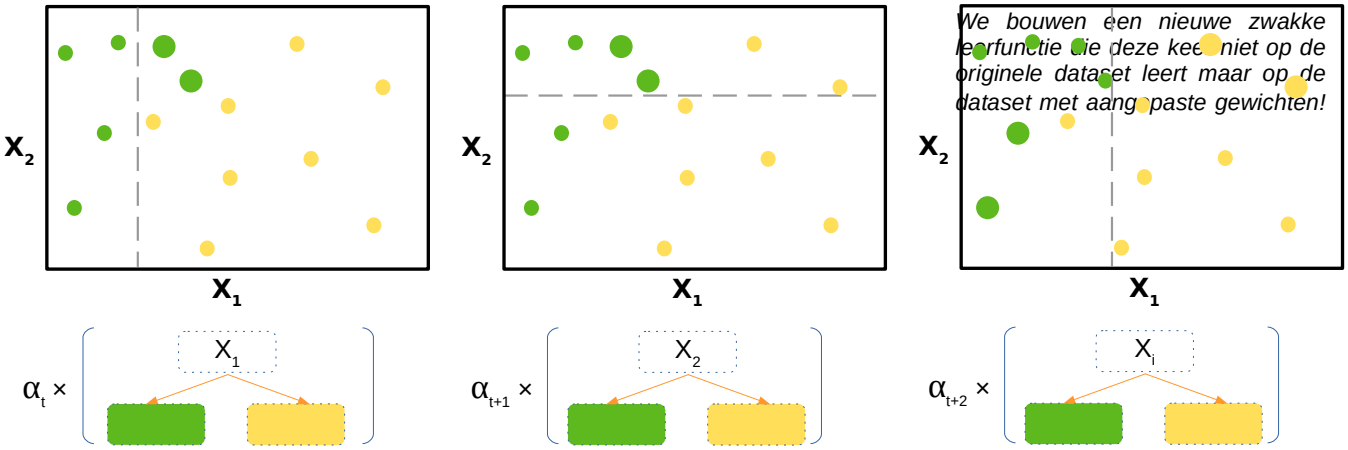
We hebben onze eerste boom!

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

*We herhalen dit proces **T** keer*

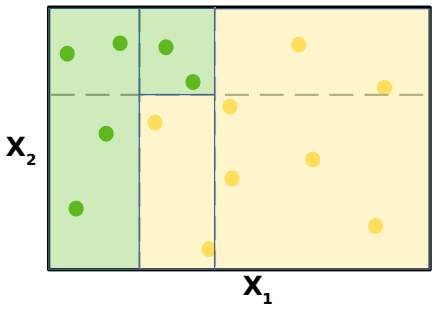
- Visualisatie van de stappen:



© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Finaal tellen we al deze voorspellingen op
- Aangezien we al deze verschillende zwakke leerfuncties hebben die elk hun eigen unieke coëfficiënten hebben, zijn we in staat iets te creëren dat eruit zal zien als een gemengd model of een ensemble van modellen.
- Dit is adaptive boosting!



© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Samenvatting:
  - AdaBoost gebruikt een ensemble van zwakke leerfuncties die serieel en traag leren
  - Bepaalde zwakke leerfuncties wegen meer door in de finale output omwille van hun α parameter
  - Elke volgende zwakke leerfunctie t is gebouwd op een op basis van zwakke leerfunctie t-1 gewogen dataset

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Intuitie van Adaptive Boosting:
  - Elke stump representeert de voorspellende kracht van één feature
  - Door deze stumps serieel te bouwen en een α parameter toe te voegen zijn we in staat om het totale belang van een feature te bepalen

© 2023 Olivier Claerbout

AdaBoost – Adaptive Boosting

- Enkele opmerkingen:
  - In tegenstelling tot random forests is overfitting wel degelijk mogelijk met Adaboost – maar er zijn veel ‘trees’ nodig om dit te doen
  - In het algemeen zal de error reeds lang gestabiliseerd zijn voor er voldoende trees zijn om te overfitten

© 2023 Olivier Claerbout

AdaBoost in praktijk

© 2023 *Olivier Claerbout*

Gradient Boosting

© 2023 *Olivier Claerbout*

Gradient Boosting

- Het idee is erg gelijkaardig aan AdaBoost, waar weak learners in serie (een weak learner leert van de fouten van de vorige weak learner) worden gecombineerd om een sterk ensemble model te maken.
- Gradient Boosting zou gebruikt kunnen worden met andere modellen, maar werkt best met decision trees.
- Elk model richt zich op het corrigeren van de fouten van het geaggregeerde model tot dat punt. Gradient Boosting gebruikt de residual error om te leren.

© 2023 Olivier Claerbout

Gradient Boosting

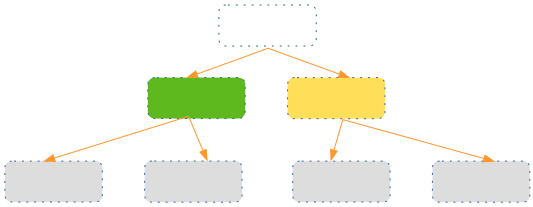
- Gradient Boosting versus AdaBoost:
  - Gradient Boosting staat grotere Trees toe dan de stumps bij Adaboost
  - De Learning Rate Coëfficiënt is dezelfde voor alle weak learners bij gradient boosting, terwijl bij Adaboost elke weak learner een eigen coëfficiënt had
  - Het serieel leren is bij gradient boosting gebaseerd op het trainen op de residuals van het vorige model

© 2023 Olivier Claerbout

Gradient Boosting

- Gradient Boosting: voorbeeld met regressie

Oppervlakte	Slaapkamers	Badkamers	Prijs
200	3	2	€ 500.000
190	2	1	€ 462.000
230	3	3	€ 565.000



We trainen een decision tree (geen stump!) op de dataset

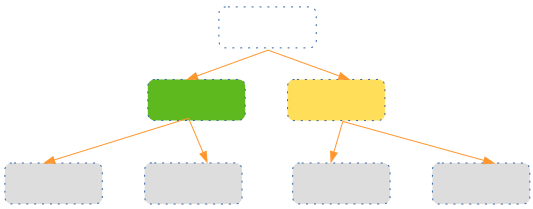
We voorspellen $\hat{y}$

© 2023 Olivier Claerbout

Gradient Boosting

- Gradient Boosting: voorbeeld met regressie

y	$\hat{y}$	e
€ 500.000	€ 509.000	– € 9.000
€ 462.000	€ 509.000	– € 47.000
€ 565.000	€ 509.000	€ 56.000



Deze € 509.000 is gewoon het gemiddelde van de huizenprijzen...

We berekenen de residual $e = y - \hat{y}$

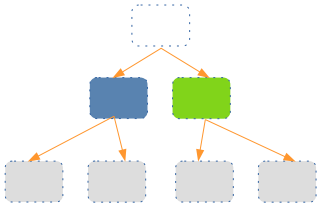
Deze residual heeft informatie omtrent de performantie van de eerste decision tree

© 2023 Olivier Claerbout

Gradient Boosting

- We bouwen een nieuw model om de **error** te voorspellen uit de *originele features*

y	ŷ	e	f1	F1 = ŷ + f1
€ 500.000	€ 509.000	– € 9.000	– € 8.000	€ 501.000
€ 462.000	€ 509.000	– € 47.000	– € 50.000	€ 459.000
€ 565.000	€ 509.000	€ 56.000	€ 50.000	€ 559.000



We updaten nu onze voorspelling met de voorspelde error erbij

We benaderen de werkelijke prijs nu beter! We leren dus uit fouten van het voorgaande model...

We kunnen dit proces blijven herhalen...

© 2023 Olivier Claerbout

Gradient Boosting

- Gradient Boosting formule:

$$F_m(x) = F_{m-1} + f_m$$

$$F_m(x) = F_{m-1} + (learning\,rate * f_m)$$

- De voorspelling van het huidige model, F_m, is de som van de voorspelling van het vorige model, F_{m-1} , en de voorspelling van de huidige weak learner of beslisboom, f_m.
- Elk nieuw model voegt dus iets toe aan de totale voorspelling.
- Het Gradient Boosting proces wordt aangepast door gebruik van een learning rate. We vermenigvuldigen dus de voorspelling van de residual met een learning rate.
- Deze learning rate is bij Gradient Boosting identiek voor alle modellen, itt Adaboost waar elke learner een eigen coëfficiënt had.

© 2023 Olivier Claerbout

Gradient Boosting

- Learning rate α:
  - Wordt niet toegepast op het eerste model wat de initiële voorspelling levert
  - Bepaalt hoe snel een model leert en zit tussen 0 en 1
  - Elke bijkomende tree wijzigt het model. Hoe sterk deze wijziging is wordt bepaald door de learning rate
  - Een kleinere learning rate zorgt voor een kleinere impact van elke individuele tree op het finale model, met een trager leerproces met meer trees als gevolg
  - Trager leren leidt vaak tot betere modellen, maar tegen een prijs: een hogere trainingstijd
  - Gradient Boosting is redelijk ongevoelig voor overfitting. Het aantal estimators mag daardoor default op een vrij hoge waarde (~100)
  - Vaak geeft een learning rate in de range van 0,1 tot 0,3 goede resultaten
  - Learning rate en n_estimators worden via grid search bepaald

© 2023 Olivier Claerbout

Gradient Boosting

- Gradient Boosting proces:
  - Initieel model: f_0
  - Train een model om de error te voorspellen:
    - $e = y - f_0$
  - Maak een nieuwe voorspelling
    - $F_1 = f_0 + \alpha f_1$
  - Herhaal indien nodig
    - $F_m = f_{m-1} + \alpha f_m$

© 2023 Olivier Claerbout

Gradient Boosting

- Voor classificatie kunnen we de logit als error metric gebruiken (zie logistieke regressie):

$$\hat{y} = \log\left(\frac{\hat{p}}{1-\hat{p}}\right) \qquad \hat{p} = \frac{1}{1+e^{-\hat{y}}}$$

© 2023 Olivier Claerbout

Gradient Boosting

- Bij Gradient Boosting:
  - Optimaliseren we een serie trees door te leren van de residuals, waardoor opeenvolgende trees worden gedwongen om de fout die voorafgaande trees hebben gemaakt te corrigeren.

© 2023 Olivier Claerbout

Gradient Boosting in praktijk!

© 2023 *Olivier Claerbout*